



PROCUREMENT & PURCHASE REQUEST MANAGEMENT SYSTEM

Lab Guide

FINAL POWER PLATFORM CAPSTONE PROJECT

Procurement & Purchase Request Management System

Project Overview

Your organization currently manages purchase requests through a combination of:

-  Emails
-  Excel files
-  Teams messages
-  Phone calls
-  Paper approvals

Employees send purchase requests to their managers manually. Managers approve them through email, and the Procurement team tracks approved requests in spreadsheets.

This creates several problems:

- ✗ Requests get lost
- ✗ Duplicate requests are created
- ✗ Approval history is difficult to track
- ✗ Employees do not know request status
- ✗ Procurement has no central queue
- ✗ Management has poor visibility
- ✗ Reporting requires manual Excel work
- ✗ Security is inconsistent

Your job is to design and build a complete:

Procurement & Purchase Request Management Platform

using Microsoft Power Platform.

Business Goal

Create a system where an employee can:

Select Products



Create Purchase Request



Add Multiple Items



Submit Request



Manager Reviews



Procurement Reviews



Request Is Processed



Employee Tracks Status

Management should also receive analytics about procurement activity.

Technologies Students Must Use

The mandatory solution must contain:

Technology	Purpose
 Dataverse	Central data platform
 Canvas App	Employee purchase request application
 Power Automate	Approval and notification automation
 Model-Driven App	Procurement administration
 Power BI	Procurement analytics

Technology

Purpose

 Dataverse Security Role-based access

 Solutions Application packaging

 ALM DEV → TEST → PROD deployment approach

Optional advanced features will use:

 Power Pages

 Copilot Studio

 Custom Connector / API

BUSINESS PERSONAS

1. Employee / Requester

The employee should be able to:

Browse Products

Create Request

Add Products to Cart

Change Quantity

Remove Products

Save Draft

Submit Request

View Own Requests

Track Status

The employee must NOT be able to approve their own request.

2. Manager

The manager should be able to:

Receive Approval Request

View Request Details

Approve

Reject

Add Comments

The manager should see only requests relevant to their approval responsibilities.

3. Procurement Officer

Procurement users should be able to:

View Approved Requests

Review Requested Items

Select Supplier

Process Purchase

Update Procurement Status

Handle Exceptions

Manage Products

Manage Suppliers

They will primarily use a:

 **Model-Driven App.**

4. System Administrator

The administrator should be able to:

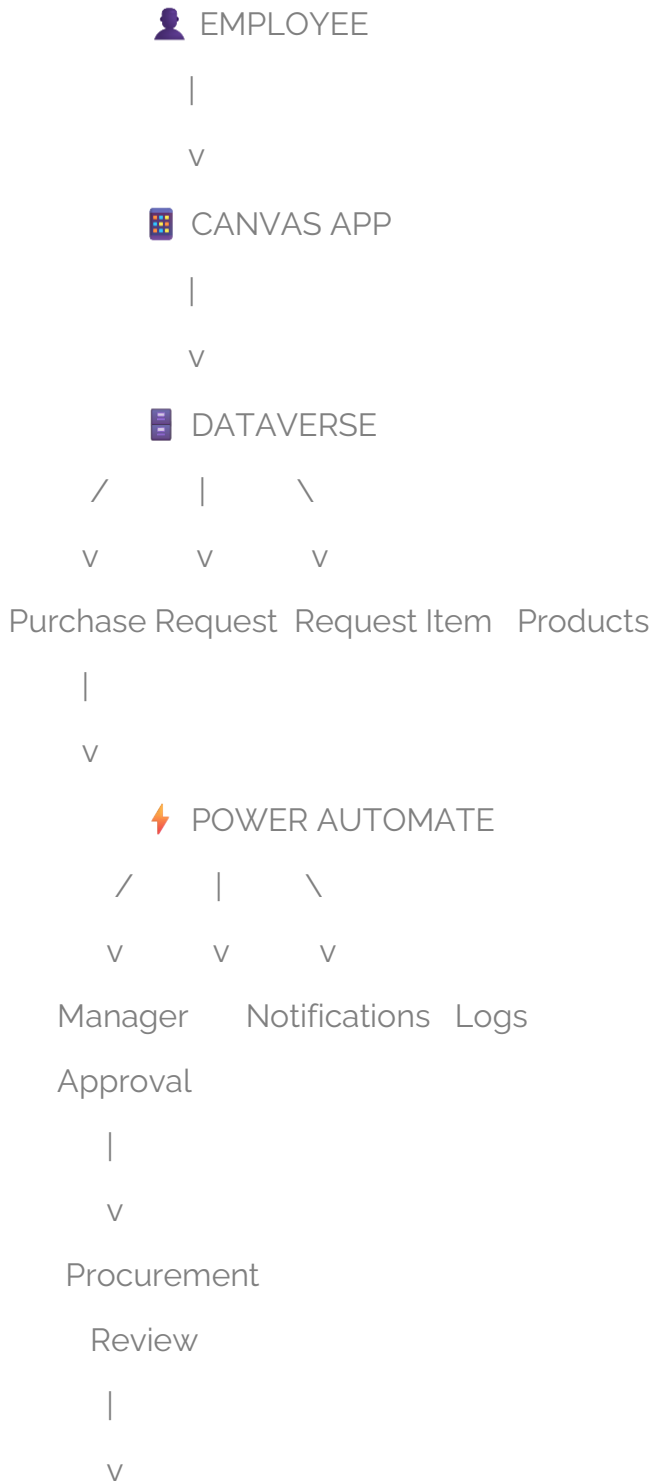
Manage all procurement records

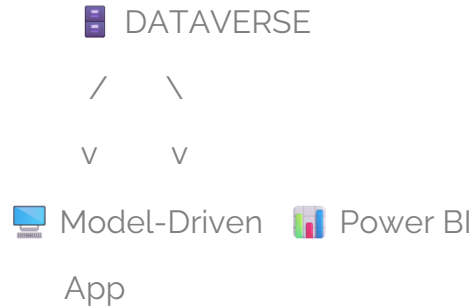
Review failed processes

Correct exceptional records

Manage configuration

 HIGH-LEVEL SOLUTION ARCHITECTURE





PART 1 — DESIGN THE DATAVERSE MODEL

Before students create the application, they must design the data model. This should be submitted as a simple ERD before development begins.

TABLE 1 — Purchase Request

This table represents the request header.

One row means:

One Purchase Request.

Recommended columns:

Column	Type	Description
Request Number	Autonumber	Unique readable request ID
Request Title	Text	Short description
Requester	Lookup → Contact/User	Employee requesting
Department	Lookup → Department	Requesting department
Request Date	Date/Time	Date created
Required Date	Date	When items are required
Business Justification	Multiline Text	Why purchase is needed
Status	Choice	Current business state
Total Amount	Currency	Total request value

Column	Type	Description
Current Approval Stage	Choice	Current approval stage
Submitted On	Date/Time	Submission time
Approved On	Date/Time	Final approval time
Procurement Notes	Multiline Text	Internal notes

REQUEST NUMBER

Students should use an Autonumber.

Example:

PR-000001

PR-000002

PR-000003

This should be easier for users to communicate than a Dataverse GUID.

REQUEST STATUS CHOICE

Create a reusable Choice called:

Purchase Request Status

Recommended values:

Draft

Submitted

Pending Manager Approval

Pending Procurement Review

Approved

Rejected

Processing

Completed

Cancelled

Failed

Students must understand that:

Status represents business state.

It should not simply represent what screen the user is currently viewing.

 TABLE 2 — Request Item

This table represents individual products inside a request.

One row means:

One requested product line.

Recommended columns:

Column	Type
Purchase Request Lookup	Purchase Request
Product	Lookup → Product
Quantity	Whole Number
Unit Price	Currency
Line Total	Currency / Formula
Notes	Text

Relationship:

Purchase Request

|

| 1

|

| *

Request Item

This is the:

Header / Line Pattern.

TABLE 3 — Product

Recommended columns:

Column	Type
Product Code	Text
Product Name	Text
Category	Choice or Lookup
Standard Price	Currency
Description	Multiline Text
Active	Yes/No
Preferred Supplier Lookup → Supplier	
Product Image	Image

Example products:

Laptop
Monitor
Keyboard
Mouse
Office Chair
Printer
Headset
Webcam

Students should create at least:

15 sample products.

TABLE 4 — Supplier

Recommended columns:

Supplier Name

Supplier Code

Email

Phone

Address

Tax Number

Status

Supplier Status:

Active

Inactive

Blocked

Pending Review

TABLE 5 — Department

Recommended columns:

Department Name

Department Code

Manager

Active

Sample data:

Finance

Human Resources

IT

Marketing

Procurement

Operations

Sales

✓ **TABLE 6 — Approval History**

This table stores permanent business approval history.

Recommended columns:

Column	Type
Purchase Request Lookup	
Approver	User
Approval Stage	Choice
Decision	Choice
Comments	Multiline Text
Decision Date	Date/Time

Example:

PR-000045

Manager Approval

Approved

Approved by Hana

18-Sep-2026

"Budget confirmed."

This table is important because Power Automate run history should NOT be the only record of a business approval.

OPTIONAL TABLE — Process Log

Advanced students can create:

Process Log

with fields:

Purchase Request

Process Name

Stage

Status

Flow Run ID

Message

Error Details

Started On

Completed On

This provides better operational troubleshooting.

FINAL RELATIONSHIP MODEL

Students should arrive at something close to:

Department

|

| 1

|

| *

Employee / User

|

| 1

|

| *

Purchase Request



Request Item Approval History



Product



Supplier

Advanced students may create a:

Supplier Product

junction table if one product can be purchased from several suppliers.

PART 2 — CREATE THE SOLUTION

Students should NOT build everything directly in the Default solution.

Create:

Solution Name:

Procurement Management

Publisher:

Student / Organization Publisher

Version:

1.0.0.0

Add all project components to this solution.

The solution should eventually contain:

Tables

Columns

Choices

Relationships

Canvas App

Model-Driven App

Flows

Security Roles

Environment Variables

Connection References

ENVIRONMENT VARIABLES

Create environment variables for configuration that may change.

Examples:

Support Email

Procurement Email

Approval Threshold

Application URL

Power BI URL

Example:

ProcurementSupportEmail

=

procurement@company.com

Do not repeatedly hardcode this email inside flows.

CONNECTION REFERENCES

Flows inside the solution should use connection references.

Example:

Approval Flow



Office 365 Outlook Connection Reference



Production Outlook Connection

This makes deployment between environments easier.

PART 3 — EMPLOYEE CANVAS APP

Create:

Procurement Request App

The app should look modern, responsive, and business-ready.

SCREEN 1 — Home

Show:

Welcome, Hana 🙌

[Create Purchase Request]

My Requests

Draft 2

Pending 3

Approved 5

Completed 12

Suggested areas:

Header

Quick actions

Request summary

Recent requests

Navigation

 SCREEN 2 — My Requests

Display the logged-in user's requests.

Gallery fields:

Request Number

Title

Date

Total Amount

Status

Example:

PR-000104

New Laptops

45,000 ETB

Pending Manager Approval

Add filters:

All

Draft

Pending

Approved

Rejected

Completed

Add search.

SEARCH CHEAT

Example:

Filter(

 'Purchase Requests',

 Requester.Email = User().Email &&

 (

 IsBlank(txtSearch.Text) ||

 StartsWith('Request Number', txtSearch.Text) ||

 StartsWith('Request Title', txtSearch.Text)

)

)

Students must check delegation warnings.

SCREEN 3 — Create Request

Fields:

Request Title

Department

Required Date

Business Justification

The initial status should be:

Draft

Then the user proceeds to:

Product Selection.

SCREEN 4 — Product Catalog

Display products using a gallery.

Example:

 Laptop Pro

25,000 ETB

 Monitor 27"

8,500 ETB

 Keyboard

1,200 ETB

Each item should have:

[Add to Cart]

SHOPPING CART COLLECTION

Create a collection called:

colCart

When a product is selected:

```
Collect(  
  colCart,  
  {  
    Product: ThisItem,  
    Quantity: 1,  
    UnitPrice: ThisItem.'Standard Price'  
  }  
)
```

PREVENT DUPLICATE PRODUCT

Better pattern:

```
If(  
  IsBlank(  
    LookUp(  
      colCart,  
      Product.Product = ThisItem.Product  
    )  
  ),  
  Collect(  
    colCart,  
    {  
      Product: ThisItem,  
      Quantity: 1,  
      UnitPrice: ThisItem.'Standard Price'  
    }  
  )
```

```
    ),  
    Notify(  
        "Product is already in your cart.",  
        NotificationType.Information  
    )  
)
```

Students may implement another correct method.

+ INCREASE QUANTITY

```
Patch(  
    colCart,  
    ThisItem,  
    {  
        Quantity: ThisItem.Quantity + 1  
    }  
)
```

- DECREASE QUANTITY

```
If(  
    ThisItem.Quantity > 1,  
    Patch(  
        colCart,  
        ThisItem,  
        {  
            Quantity: ThisItem.Quantity - 1  
        }  
    )  
)
```

)
)

REMOVE ITEM

Remove(
 colCart,
 ThisItem
)

CART TOTAL

Sum(
 colCart,
 Quantity * UnitPrice
)

Display:

Subtotal: 63,400 ETB

Items: 4

SCREEN 5 — Review Request

Before submission, show:

Request Title

Department

Required Date

Justification

Products

Quantities

Line Totals

Total Amount

Example:

Laptop × 2 50,000

Monitor × 1 8,500

Mouse × 2 1,600

Total 60,100

Buttons:

[Back]

[Save Draft]

[Submit Request]

SUBMISSION VALIDATION

Before submission, verify:

Request title exists

Department selected

Required date selected

Justification entered

Cart contains at least one product

All quantities > 0

Example:

If(

 IsBlank(txtTitle.Text) ||

 IsBlank(cmbDepartment.Selected) ||

 IsEmpty(colCart),

```

Notify(
    "Complete all required information.",
    NotificationType.Error
),

/* Submit */
)

```

PARENT / CHILD SUBMISSION

This is one of the most important parts of the project.

Students must understand:

Purchase Request

=

Parent

Request Items

=

Children

Create the parent first.

Then create the children.

STEP 1 — CREATE PARENT

Example:

```

Set(
    varRequest,

```

```

Patch(
    'Purchase Requests',
    Defaults('Purchase Requests'),
    {
        'Request Title': txtTitle.Text,
        Department: cmbDepartment.Selected,
        'Business Justification': txtJustification.Text,
        'Required Date': dpRequired.SelectedDate,
        'Total Amount':
            Sum(
                colCart,
                Quantity * UnitPrice
            ),
        Status:
            'Purchase Request Status'.Submitted
    }
)
)
varRequest now contains the newly created Purchase Request.

```

STEP 2 — CREATE CHILD RECORDS

```

ForAll(
    colCart As cartItem,
    Patch(
        'Request Items',
        Defaults('Request Items'),

```

```

    {
        'Purchase Request': varRequest,
        Product: cartItem.Product,
        Quantity: cartItem.Quantity,
        'Unit Price': cartItem.UnitPrice,
        'Line Total':
            cartItem.Quantity *
            cartItem.UnitPrice
    }
)
)

```

STEP 3 — COMPLETE SUBMISSION

After successful submission:

```
Clear(colCart);
```

```
Notify(
```

```
    "Purchase request submitted successfully.",
```

```
    NotificationType.Success
```

```
);
```

```
Navigate(scrMyRequests)
```

ADVANCED REQUIREMENT — SUBMISSION LOCK

Students should prevent:

Double Click

→ Duplicate Request

Use:

varSubmitting

Example:

Set(varSubmitting, true)

Submit button:

If(

varSubmitting,

DisplayMode.Disabled,

DisplayMode.Edit

)

After processing:

Set(varSubmitting, false)

Better students should also think about server-side duplicate prevention.

⚡ PART 4 — POWER AUTOMATE APPROVAL

Create:

Purchase Request Approval Flow

Trigger:

When Purchase Request is submitted

Prefer a Dataverse trigger that runs only for the meaningful state transition rather than every modification.

Flow architecture:

Request Submitted

↓

Validate Request

↓

Get Manager



Update Status

Pending Manager Approval



Start Approval



Approve / Reject

MANAGER APPROVAL

Approval should include:

Request Number

Requester

Department

Total Amount

Business Justification

Requested Products

Approval outcomes:

Approve

Reject

IF APPROVED

Manager Approves



Create Approval History



Status =

Pending Procurement Review



Notify Procurement

IF REJECTED

Manager Rejects



Create Approval History



Status = Rejected



Save Comments



Notify Employee

PROCUREMENT REVIEW

You can implement procurement review using either:

Another Power Automate Approval

or:

Model-Driven App Review

Recommended final-project approach:

Manager Approval



Procurement Review Queue



Procurement Officer



Approve for Processing



FINAL STATUS FLOW

A successful request might follow:

Draft



Submitted



Pending Manager Approval



Pending Procurement Review



Approved



Processing



Completed

Rejected:

Pending Manager Approval



Rejected



NOTIFICATIONS

Send meaningful notifications.

Employee:

Your request PR-000104 was submitted.

Manager:

Purchase request PR-000104 requires your approval.

Employee after approval:

PR-000104 has been approved by your manager.

Employee after rejection:

PR-000104 was rejected.

Reason:

Budget unavailable.

Do not send technical messages such as:

Flow execution completed successfully.

ERROR HANDLING

Use:

TRY Scope

|

v

Main Processing

CATCH Scope

|

|—— Set Status = Failed

|—— Create Process Log

|—— Notify Support

FINALLY Scope

|

└── Final Audit / Cleanup

Configure:

Run After

correctly.

RETRY STRATEGY

Temporary failures may include:

HTTP 500

HTTP 502

Temporary network failure

Rate limitation

These may be retried.

Do not endlessly retry:

Invalid credentials

Missing required fields

Permission denied

Invalid record

PART 5 — MODEL-DRIVEN PROCUREMENT ADMIN APP

Create:

Procurement Administration

The Model-Driven App should contain:

Purchase Requests

Request Items

Products

Suppliers
Departments
Approval History
Process Logs

MODEL-DRIVEN NAVIGATION

Example:

Procurement



IMPORTANT VIEWS

Students should create at least:

My Procurement Queue

Pending Procurement Review

High Value Requests

Approved Requests

Rejected Requests

Requests Requiring Attention

Completed Requests

PURCHASE REQUEST FORM

Recommended organization:

SUMMARY

Request Number

Requester

Department

Status

Total Amount

REQUEST DETAILS

Required Date

Business Justification

ITEMS

Request Item Subgrid

APPROVAL

Current Approval Stage

Approval History Subgrid

PROCUREMENT

Supplier

Procurement Notes

Processing Status



PART 6 — SECURITY

Create Dataverse security roles.



REQUESTER ROLE

Should approximately allow:

Purchase Request

Create

Read Own

Write Own Draft

Request Item

Create

Read Own Related

Write Own Related

Requester should not have broad:

Delete all

Approve

Administer suppliers
permissions.

MANAGER ROLE

Should have access required to:

- Read approval-related requests
- Review required information
- Participate in approval

PROCUREMENT ROLE

Should allow:

- Read Purchase Requests
- Update Procurement Fields
- Manage Products
- Manage Suppliers
- Read Approval History

ADMIN ROLE

Administrative users receive broader privileges required for operational support.

IMPORTANT SECURITY RULE

Students must understand:

Filter(

Requests,

Requester.Email = User().Email

)

is:

UX filtering.

It is NOT sufficient authorization.

Actual security must come from:

Dataverse.

PART 7 — POWER BI PROCUREMENT DASHBOARD

Create a Power BI report connected to the procurement data.

Suggested pages:

1. Executive Overview
2. Department Analysis
3. Product Analysis
4. Approval Performance
5. Procurement Operations

EXECUTIVE KPIs

Include:

Total Requests

Total Requested Amount

Approved Amount

Rejected Amount

Pending Requests

Completed Requests

Approval Rate

Average Request Value

DAX CHEAT

Total Requests:

Total Requests =

DISTINCTCOUNT(

'Purchase Request'[Request Number]

)

Total Requested Amount:

Total Requested Amount =

SUM(

'Purchase Request'[Total Amount]

)

Approved Requests:

Approved Requests =

CALCULATE(

[Total Requests],

'Purchase Request'[Status] = "Approved"

)

Approval Rate:

Approval Rate =

DIVIDE(

[Approved Requests],

[Total Requests]

)

REQUIRED VISUALS

Include at least:

KPI Cards

Requests by Department

Requested Amount by Department

Requests by Status

Top Requested Products

Monthly Request Trend

Approval Rate

Average Approval Duration

Add slicers such as:

Date

Department

Status

Requester

Supplier

★ **BONUS 1 — POWER PAGES**

Advanced students can build a supplier portal.

Scenario:

Supplier



Power Pages



Login



View Purchase Opportunities



Submit Supplier Information

Possible features:

Supplier Registration

Supplier Profile

Document Upload

Order Status

Students must implement proper:

Authentication

Web Roles

Table Permissions

BONUS 2 — COPILOT STUDIO

Create:

Procurement Assistant

Example questions:

What is our procurement policy?

How do I create a purchase request?

What information do I need?

What is the difference between draft and submitted?

Add knowledge from:

Procurement Policy

FAQ

Purchase Request Guide

Advanced tool:

"Check my request PR-000104."

Agent:

User



Copilot Studio



Tool / Flow



Dataverse



Request Status



Agent Response

BONUS 3 — CUSTOM CONNECTOR

Integrate with an external supplier API.

Example:

Power Automate



Custom Connector



Supplier API



Check Product Price

Students should demonstrate understanding of:

GET

POST

JSON

Authentication

Status Codes

PART 8 — ALM

The solution must follow:

DEV

↓

TEST

↓

PROD

Students must not treat production as the development environment.

SOLUTION CONTENT

Before deployment confirm:

☐ Dataverse Tables

☐ Choices

☐ Relationships

☐ Canvas App

☐ Model-Driven App

☐ Cloud Flows

☐ Security Roles

☐ Environment Variables

☐ Connection References

Version:

1.0.0.0

SOLUTION CHECKER

Run:

Solution Checker.

Students must review warnings and explain important findings.

They do not receive full credit simply for running the tool.

They must demonstrate:

Issue



Understand



Fix or justify

PART 9 — TESTING REQUIREMENTS

Students must submit a test sheet.

Required testing categories:

Functional

Integration

Security

Performance

UAT

Regression

Deployment

EXAMPLE TEST CASE

ID	Scenario	Expected Result
TC01	Create valid request	Draft created
TC02	Submit without product	Validation shown
TC03	Submit valid request	Approval begins
TC04	Manager approves	Status updated
TC05	Manager rejects	Request rejected
TC06	Employee opens another user's record	Access denied
TC07	Double-click Submit	No duplicate request
TC08	Flow fails	Error logged
TC09	Procurement completes request	Status Completed
TC10	Power BI refreshes	New transaction appears

Students should create additional tests.

Minimum:

15 test cases.

FAILURE SCENARIOS STUDENTS MUST TEST

Test what happens when:

Manager is missing

Approval is rejected

Email fails

Product is inactive

Request has no items

User submits twice

Connection fails

Unauthorized user accesses record

Flow fails halfway

Incorrect quantity is entered

This is where students demonstrate production thinking.

UAT

Students should ask another student to behave as:

Employee

Manager

Procurement Officer

The tester should not be the original developer.

Collect feedback about:

Navigation

Clarity

Performance

Errors

Status terminology

Overall usability

PART 10 — PRODUCTION READINESS

Before final submission students must verify:

- ☐ No major formula errors
- ☐ No unnecessary delegation warnings
- ☐ No hardcoded production configuration
- ☐ Security roles tested
- ☐ Flows have error handling
- ☐ Duplicate submission protected

- ☐ Connections work
 - ☐ Environment variables configured
 - ☐ Approval history stored
 - ☐ Power BI refresh works
 - ☐ Solution Checker reviewed
 - ☐ App Checker reviewed
-

FINAL STUDENT DELIVERABLES

Every student or team must submit:

1. Architecture Diagram
2. Dataverse ERD
3. Procurement Solution
4. Canvas App
5. Model-Driven App
6. Approval Flow
7. Error Handling Flow Design
8. Power BI Report
9. Security Role Design
10. Testing Document
11. Deployment Checklist
12. User Guide
13. Technical Documentation
14. Final Presentation / Demo

Bonus components should be clearly marked.

FINAL DEMO

Recommended demo length:

15–20 minutes.

Students should demonstrate the solution as a real business system.

DEMO STORY

1 Business Problem

Explain:

What problem existed?

Why was Power Platform selected?

2 Architecture

Show:

Canvas App

Dataverse

Power Automate

Model-Driven App

Power BI

and explain each component's responsibility.

3 Employee Demo

Perform:

Login

↓

Create Request

↓

Add Products

↓

Review

↓

Submit

4 Approval Demo

Show:

Manager receives approval

↓

Approve / Reject

↓

Dataverse updates

5 Procurement Demo

Open the Model-Driven App.

Show:

Procurement Queue

Request Details

Items

Approval History

Status Management

6 Analytics Demo

Show Power BI.

Explain at least three insights.

For example:

IT has the highest requested amount.

Laptop requests represent 34% of spending.

Average approval duration is 1.6 days.

7 Security

Explain:

What can Requester access?

What can Procurement access?

How is unauthorized access prevented?

8 ALM

Explain:

DEV

↓

TEST

↓

PROD

Show the solution package and configuration strategy.

🏆 CAPSTONE GRADING RUBRIC

Total:

100 Points

Area

Points



Dataverse Data Model

15

Area	Points
 Canvas App UX & Functionality	15
 Power Automate & Approvals	15
 Model-Driven Admin App	10
 Security	10
 Power BI	10
 Testing & Error Handling	10
 ALM & Solution Quality	5
 Documentation	5
 Final Demo	5
TOTAL	100

BONUS POINTS

Optional:

Bonus	Points
 Power Pages Supplier Portal	+5
 Copilot Studio Agent	+5
 Custom Connector	+5
 Excellent Advanced Architecture	+5

Maximum:

120 / 100

Bonus should reward genuine technical quality, not decorative features.

STUDENT CHEAT SHEET

DATAVERSE

Choice

Use for:

Small fixed list

Examples:

Draft

Submitted

Approved

Rejected

Lookup

Use for:

Relationship to another business entity

Examples:

Department

Product

Supplier

Requester

RELATIONSHIP CHEAT SHEET

One Purchase Request



Many Request Items

One Product



Many Request Items

One Purchase Request



Many Approval History Records

Lookup goes on the:

Many side.

POWER FX CHEAT SHEET

Navigate

```
Navigate(scrMyRequests)
```

Back

```
Back()
```

Notification

```
Notify(  
    "Request submitted.",  
    NotificationType.Success  
)
```

Global Variable

```
Set(  
    varSelectedRequest,  
    ThisItem  
)
```

Context Variable

```
UpdateContext(  
    {  
        locLoading: true  
    }  
)
```

Clear Collection

```
Clear(colCart)
```

Filter

```
Filter(  
    'Purchase Requests',  
    Status = 'Purchase Request Status'.Draft  
)
```

LookUp

```
LookUp(  
    Products,  
    'Product Code' = "PRD-001"  
)
```

Patch Existing Record

```
Patch(  
    'Purchase Requests',  
    varSelectedRequest,
```

```
{  
    'Request Title': txtTitle.Text  
}  
)
```

Patch New Record

```
Patch(  
    'Purchase Requests',  
    Defaults('Purchase Requests'),  
    {  
        'Request Title': txtTitle.Text  
    }  
)
```

Error Handling

```
IfError(  
    Patch(  
        'Purchase Requests',  
        Defaults('Purchase Requests'),  
        {  
            'Request Title': txtTitle.Text  
        }  
    ),
```

```
    Notify(  
        "Unable to create request.",  
        NotificationType.Error
```

)
)

POWER AUTOMATE CHEAT SHEET

Reliable Architecture

TRIGGER

↓

VALIDATE

↓

TRY

↓

PROCESS

↓

CATCH

↓

FINALLY

Configure Run After

Use:

Succeeded

Failed

Skipped

Timed Out

Retry

Use retry for:

Temporary network error

HTTP 500

HTTP 502

Temporary API failure

Do not rely on retry for:

Invalid password

Missing data

Permission denied

Invalid endpoint

✅ APPROVAL CHEAT SHEET

Start and Wait for an Approval



Outcome

/ \

Approve Reject

| |

Update Update

Status Status

Always store important approval decisions in Dataverse.

📊 POWER BI CHEAT SHEET

Total Amount

Total Amount =

SUM(

 'Purchase Request'[Total Amount]

)

Total Requests

Total Requests =
COUNTROWS(
 'Purchase Request'
)

Approved Requests

Approved Requests =
CALCULATE(
 [Total Requests],
 'Purchase Request'[Status] = "Approved"
)

Safe Division

Approval Rate =
DIVIDE(
 [Approved Requests],
 [Total Requests]
)

SECURITY CHEAT SHEET

Remember:

Hidden Button

≠

Security

and:

Canvas Filter

≠

Security

Real authorization comes from:

Dataverse Security Roles

Ownership

Teams

Business Units

Sharing

ALM CHEAT SHEET

Recommended architecture:

DEV

Unmanaged Solution

↓

Export Managed

↓

TEST

↓

PROD

Use:

Environment Variables

for:

URLs

Email addresses

Configuration

Environment-specific values

Use:

Connection References

for solution-aware connector connections.

TROUBLESHOOTING CHEAT SHEET

Canvas App

Check:

App Checker

Monitor

Variables

Collections

Delegation

Connector calls

Power Automate

Check:

Run History

Failed Action

Inputs

Outputs

Connection

Error Code

Solution

Check:

Solution Checker

Dependencies

Version

Connection References

Environment Variables

COMMON STUDENT MISTAKES

Avoid:

- ✗ Developing directly in production
- ✗ Putting everything in one Canvas screen
- ✗ Loading all Dataverse records into collections
- ✗ Treating UI filters as security
- ✗ Hardcoding email addresses everywhere
- ✗ Using text instead of relationships
- ✗ Saving product lists in one text field
- ✗ Creating duplicate product lines
- ✗ Starting approval before lines are created
- ✗ Navigating immediately after SubmitForm()
- ✗ Ignoring OnSuccess / OnFailure
- ✗ Allowing double submission
- ✗ Ignoring failed flows
- ✗ Using personal connections in production
- ✗ Keeping important approval history only in Power Automate
- ✗ Giving everyone System Administrator

RECOMMENDED 4-DAY EXECUTION PLAN

This capstone matches Days 29–32 of the bootcamp.

Day 29 — Architecture & Dataverse

Students complete:

Business Process

Personas

ERD

Tables

Columns

Relationships

Choices

Security Design

Solution Creation

End-of-Day Deliverable

ERD

+

Dataverse Model

+

Architecture Diagram

Day 30 — Canvas App

Students complete:

Home

My Requests

Create Request

Product Catalog

Shopping Cart

Review

Parent / Child Submission

Validation

Responsive UI

End-of-Day Deliverable

A working employee procurement app.

Day 31 — Automation, Admin & Analytics

Students complete:

Approval Flow

Approval History

Notifications

Error Handling

Model-Driven App

Power BI Report

End-of-Day Deliverable

The complete integrated business process.

Day 32 — Test, Deploy & Present

Students complete:

Functional Testing

Security Testing

UAT

Solution Checker

Versioning

Deployment

Smoke Test

Documentation

Final Demo

End-of-Day Deliverable

 **Production-ready capstone presentation and demonstration.**

🏁 FINAL STUDENT CHALLENGE

Your final solution should demonstrate that you can think as more than a:

Power Apps Developer

You should demonstrate that you can think as a:

 **Power Platform Solution Architect.**

That means considering:

 Business Requirements

 Data Architecture

 User Experience

 Automation

 Security

 Analytics

 Testing

 ALM

 Governance

 Production Operations

A successful project is not the one with the most screens or flows.

The successful project is the one where:

- ✓ The data model is correct.
- ✓ The user journey is simple.
- ✓ The automation is reliable.
- ✓ The data is secure.
- ✓ The solution can be deployed.
- ✓ Another team can support it.